



**Instituto Superior
de Engenharia**

Politécnico de Coimbra

Cablagem Estruturada

Planeamento de Rede em 3 edifícios de um campus Universitário

2024-2025

Filipe Gomes
João Rodrigues

2022144673
2022142654

Licenciatura em Engenharia Informática
28 de maio de 2025

Capítulo 1

Introdução

O presente relatório documenta o desenvolvimento do Trabalho Prático de Sistemas Operativos 2, cujo objetivo é a implementação de um jogo de palavras com comunicação entre processos, memória partilhada, sincronização e utilização do Registry no sistema operativo Windows. O projeto segue rigorosamente o enunciado fornecido pela unidade curricular e foi desenvolvido integralmente em linguagem C.

Capítulo 2

Objetivos do projeto

- Implementar comunicação entre processos usando named pipes.
- Utilizar memória partilhada para letras visíveis.
- Garantir sincronização com mutexes.
- Implementar um BOT com tempo de reação aleatório.
- Usar o Registry para guardar parâmetros dinâmicos (MAXLETRAS e RITMO).
- Desenvolver comandos para jogador e árbitro.
- Controlar a entrada, saída, pontuação e eliminação de jogadores.
- Comunicar eventos entre os jogadores.

Capítulo 3

Arquitetura geral do Trabalho

O trabalho é composto por:

- Árbitro: Processo central que gere os jogadores, valida palavras, pontuações, e envia letras.
- Jogador (JogoUI): Processo que recebe letras, envia palavras e comanda interações.
- BOT: Processo automático que tenta formar palavras válidas usando as letras visíveis.

As comunicações entre os processos são feitos com: Named pipes (bidirecionais), funciona tanto para JogoUI -> Arbitro como Arbitro->JogoUI e Bot e memória partilhada para letras visíveis, com proteção por mutex.

No sistema existem 2 tipos de named pipes com funcionalidades distintas:

- `\\.\pipe\Pipe`: criado pelo árbitro, serve exclusivamente como ponto de entrada para os jogadores e receber tipos de comandos.
- `\\.\pipe\Pipe_Comando_<username>`: criado pelo processo jogador ou BOT, é um pipe persistente e bidirecional usado para a troca de mensagens durante o jogo. O árbitro conecta-se a este pipe para enviar e receber comandos, como exclusão, mensagens de letras, aviso de liderança, entre outros.

Capítulo 4

Desenvolvimento e Tecnologias usadas

- Linguagem: C (WinAPI)
- Conceitos: Threads, Named Pipes, Mutexes, Memória Partilhada, `HKEY_CURRENT_USER\Software\TP_S02`
- Threads: Uma thread por jogador para escutar comandos do árbitro, thread para geração de letras, thread para comandos do árbitro.
- Sincronização: O projeto utiliza o mutex global denominado "Global geral" para garantir exclusão mútua em zonas partilhadas entre processos. O mutex global denominado `Global\geral` é utilizado por todos os processos (árbitro, jogoui e bot), assegurando que apenas um processo manipule as letras de cada vez, evitando condições de corrida e corrupção de dados.

Capítulo 5

Evolução do Projeto por passos

- 1 Estrutura inicial, comunicação via named pipes, limite de jogadores.
- 2 Comandos :sair, :jogs, listar, excluir, encerrar. Remoção de jogadores.
- 3 Início da geração de letras aleatórias.
- 4 Validação de palavras e letras visíveis, implementação de letras visíveis no JogoUI.
- 5 Substituição definitiva de pipes por memória partilhada para letras.
- 6 Mensagens de notificação para todos os JogoUI (entrada, saída, liderança), mensagem de vencedor. BOT com Sleep aleatório entre 5-30s. Integração final do Registry.

Capítulo 6

Estruturas de Dados

```
typedef struct
{
    char tipo[20];
    char username[30];
    char palavra[30];
} mensagem;
```

Figura 6.1: Estrutura mensagem

Estrutura usada para toda a comunicação entre os processos (Jogador/Bot <-> Árbitro). Contém o tipo da ação (:pont, :jogs, palavra, etc.), o nome do jogador e, se aplicável, a palavra enviada. Esta estrutura permite um modelo de comunicação simples e extensível por Named Pipes.

```
typedef struct
{
    HANDLE hpipe;
    mensagem msg_thread;
} infoThread;
```

Figura 6.2: Estrutura auxiliar

Estrutura auxiliar para passar múltiplos parâmetros às threads que processam pedidos de jogadores no árbitro. Envia o HANDLE do pipe de comunicação e a mensagem recebida. Esta estrutura foi usada pois uma vez que o Readfile já tinha sido feito uma vez na função principal , tentar fazer uma nova leitura dentro da thread que processa os jogadores falhava.

Capítulo 7

Tabela de Requisitos implementados

ID	Requisito	Estado
1	Comunicação entre árbitro e jogoui via named pipes	Implementado
2	Comunicação entre árbitro e bot via named pipes	Implementado
3	Comunicação entre jogadores (avisos de entrada, saída, etc.)	Implementado
4	Memória partilhada para letras visíveis com mutex "Global\geral"	Implementado
5	Geração de letras com RITMO controlado e substituição da mais antiga	Implementado
6	MAXLETRAS e RITMO guardados no Registry	Implementado
7	Rejeição de nomes repetidos	Implementado
8	Comandos do jogador: <code>:sair</code> , <code>:jogs</code> , <code>:pont</code>	Implementado
9	Comandos do árbitro: <code>listar</code> , <code>excluir</code> , <code>encerrar</code> , <code>acelerar</code> , <code>travar</code> , <code>iniciarbot</code>	Implementado
10	Validação de palavras com letras visíveis	Implementado
11	Pontuação correta com penalização	Implementado
12	Remoção de letras visíveis usadas em palavras	Implementado
13	Aviso quando jogador passa a liderar	Implementado
14	BOT funcional com tempo aleatório de reação	Implementado
15	Mensagem de vencedor antes de encerrar	Implementado
16	Painel visual (opcional)	Não Implementado

Tabela 7.1: Lista de Requisitos Implementados

7.1 Justificação da Não Implementação do Painel Visual

O painel visual foi considerado opcional no enunciado do trabalho. Dado o foco prioritário nas funcionalidades obrigatórias e o nosso tempo disponível para o desenvolvimento, essa funcionalidade acabou por não ser integrada.

Capítulo 8

Diagrama de Arquitetura

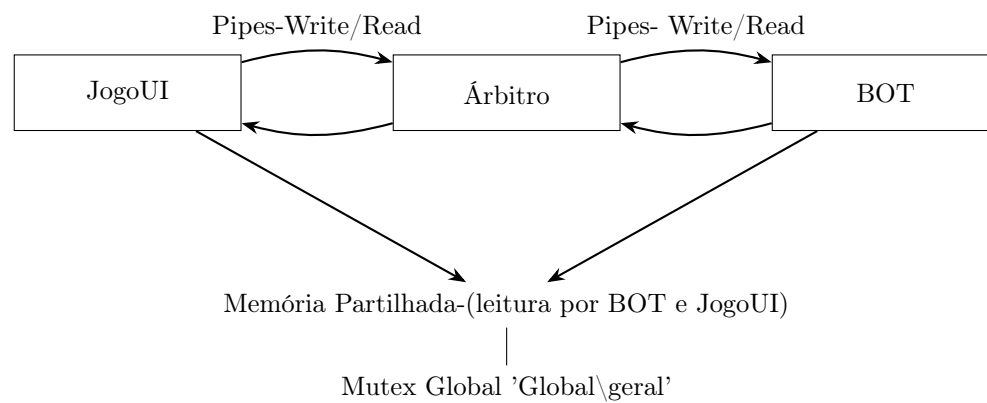


Figura 8.1: Diagrama de comunicação entre processos

Bibliografia

- [1] Enunciado oficial TP SO2 - 2024/25 (versão 6)
- [2] Documentação da Microsoft: WinAPI Named Pipes, CreateThread, CreateMutex, Registry API.
<https://learn.microsoft.com/en-us/windows/win32/api/>
- [3] Chat GPT